NOVASHOP / TD2

Oumayma Ennamous - 5A - Architecture logicielle / IaC

01 / Architecture et reproductibilité

Objectif : déployer une application 3-tiers sur une VM K3s dans OpenStack.

TERRAFORM
VM, sécurité, IP flottante

→

ANSIBLE
Ubuntu, K3s, kubeconfig

→

HELM
3 charts indépendants

VM k3s-ennamous - namespace td2

Navigateur > FIP:80 > Traefik > nginx x2 > API Python x2 > PostgreSQL

Administration : Helm / kubectl > tunnel SSH:22 > API K3s:6443

SG : TCP 22/80/443 ; 6443 non ouvert. FIP : 172.24.4.192 ; IP privée : 10.10.10.33.

Organisation du dépôt et séparation des responsabilités

```
terraform/  provider, variables, data sources, VM, SG, FIP, outputs
ansible/    playbook et rôles : préparation Ubuntu, K3s et kubeconfig*
helm/       postgres/ (StatefulSet + PVC), backend/, frontend/
docs/       captures, preuves.md et reflexion.md
```

Sorties Terraform - capture source p. 6

```
Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

Outputs:

floating_ip = "172.24.4.192"
private_ip = "10.10.10.33"
ssh_command = "ssh ubuntu@172.24.4.192"
vm_name = "k3s-ennamous"
stack@openstack-devstack:~/td2/terraform$
```

Ansible : deux passages - captures sources p. 9 et p. 16

```
Passage 1 : ok=14 changed=1 unreachable=0 failed=0 skipped=1
Passage 2 : ok=14 changed=0 unreachable=0 failed=0 skipped=1
```



Extraits recadrés : premier passage



Extrait recadré : passage sans changement

Idempotence et persistance

```
terraform plan : No changes. Your infrastructure matches the configuration.
```

Les captures montrent un état Terraform stable et un second passage Ansible sans changement. Les trois upgrades Helm atteignent la révision 2, avec le statut deployed (p. 16). PostgreSQL utilise un PVC Bound de 1 Gi, local-path, et PGDATA dans un sous-répertoire.

* Le playbook référence kubeconfig, absent du dépôt consulté. Les preuves attestent l'exécution du TD ; une reproduction exige de récupérer ce rôle. Les sorties détaillées restent dans docs/captures/.

02 / Validation de l'application

Preuves originales extraites du document de travail TD2 de 17 pages.

```
helm list -n td2 ; kubectl -n td2 get pods,svc,statefulset,ingress,pvc
```

```
stack@openstack-devstack:~/td2/helm$ cd ~/td2/terraform
terraform output -raw floating_ip
172.24.4.192stack@openstack-devstack:~/td2/terraform$
stack@openstack-devstack:~/td2/terraform$ export FIP=$(terraform output -raw floating_ip)
stack@openstack-devstack:~/td2/terraform$ echo "URL : http://$FIP"
URL : http://172.24.4.192
stack@openstack-devstack:~/td2/terraform$ helm list -n td2
NAME            NAMESPACE       REVISION        UPDATED                                 STATUS          CHART           APP VER
SION
backend         td2             1               2026-10-12 12:51:19.372923073 +0000 UTC deployed        backend-0.1.0   1.16.0

frontend        td2             1               2026-10-12 13:04:58.117654295 +0000 UTC deployed        frontend-0.1.0  1.16.0

postgres        td2             1               2026-10-12 12:40:04.273795315 +0000 UTC deployed        postgres-0.1.0  1.16.0

stack@openstack-devstack:~/td2/terraform$ kubectl -n td2 get pods,svc,statefulset,ingress,pvc
NAME                            READY   STATUS    RESTARTS   AGE
pod/backend-665d65c888-l44ps    1/1     Running   0          16m
pod/backend-665d65c888-lgm8l    1/1     Running   0          16m
pod/frontend-f6b9c579b-nhfbb    1/1     Running   0          2m54s
pod/frontend-f6b9c579b-zwp9c    1/1     Running   0          2m54s
pod/postgres-0                  1/1     Running   0          27m

NAME               TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)    AGE
service/backend    ClusterIP   10.43.193.196   <none>        5000/TCP   16m
service/frontend   ClusterIP   10.43.28.87     <none>        80/TCP     2m54s
service/postgres   ClusterIP   None            <none>        5432/TCP   27m

NAME                        READY   AGE
statefulset.apps/postgres   1/1     27m

NAME                                    CLASS     HOSTS   ADDRESS       PORTS   AGE
ingress.networking.k8s.io/frontend      traefik   *       10.10.10.33   80      2m54s

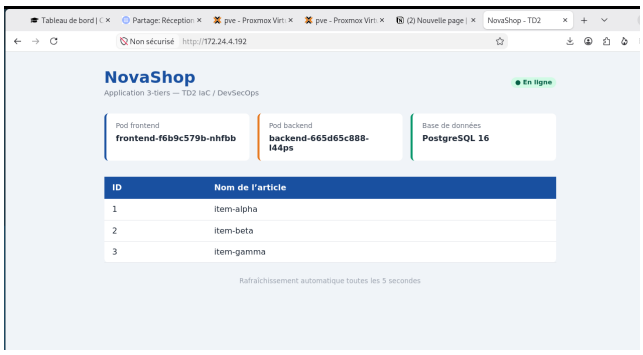
NAME                                      STATUS   VOLUME                                     CAPACITY   ACCESS MODES   STORAGECL
ASS   VOLUMEATTRIBUTESCLASS   AGE
persistentvolumeclaim/data-postgres-0   Bound    pvc-31c59890-6686-41a1-bc0e-7baa0fd9929c   1Gi        RWO            local-pat
h     <unset>                 27m
stack@openstack-devstack:~/td2/terraform$
```

Source p. 13 : 3 releases deployed ; 5 pods Running ; Ingress Traefik ; PVC Bound.

Accès HTTP, répartition frontend et interface NovaShop

```
stack@openstack-devstack:~/td2/terraform$ kubectl -n td2 get endpoints
Warning: v1 Endpoints is deprecated in v1.33+; use discovery.k8s.io/v1 EndpointSlice
NAME       ENDPOINTS                       AGE
backend    10.42.0.16:5000,10.42.0.17:5000   17m
frontend   10.42.0.23:80,10.42.0.24:80       3m27s
postgres   10.42.0.15:5432                 28m
stack@openstack-devstack:~/td2/terraform$ curl -I "http://$FIP" | head -3
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
  0  4574    0     0    0     0      0      0 --:--:-- --:--:-- --:--:--     0
HTTP/1.1 200 OK
Accept-Ranges: bytes
Content-Length: 4574
stack@openstack-devstack:~/td2/terraform$ for i in $(seq 1 6); do
  curl -sI "http://$FIP" | grep -i '^X-Pod-Name'
done
X-Pod-Name: frontend-f6b9c579b-nhfbb
X-Pod-Name: frontend-f6b9c579b-zwp9c
X-Pod-Name: frontend-f6b9c579b-nhfbb
X-Pod-Name: frontend-f6b9c579b-zwp9c
X-Pod-Name: frontend-f6b9c579b-nhfbb
X-Pod-Name: frontend-f6b9c579b-zwp9c
stack@openstack-devstack:~/td2/terraform$ curl -s "http://$FIP/api/items" | python3 -m json.tool
{
    "pod": "backend-665d65c888-lgm8l",
    "items": [
        {
            "id": 1,
            "name": "item-alpha"
        },
        {
            "id": 2,
            "name": "item-beta"
        },
        {
            "id": 3,
            "name": "item-gamma"
        }
    ]
}
stack@openstack-devstack:~/td2/terraform$
```

Source p. 13 : HTTP 200, X-Pod-Name et JSON.



Interface capturée (source p. 14).


```
{"pod":"backend-665d65c888-lgm8l",
 "items":[
  {"id":1,"name":"item-alpha"},
  {"id":2,"name":"item-beta"},
  {"id":3,"name":"item-gamma"}]}
```


Les deux noms frontend observés prouvent la répartition ; le JSON est transcrit à droite.

03 / Réponses aux 5 questions

Synthèse des choix techniques et du périmètre de destruction.

Q1 - Pourquoi des data sources pour asl-net ?

Le réseau asl-net existe depuis le TD1. Une data source lit ses informations pour connecter la VM sans le recréer ni le gérer. Une resource en gérerait le cycle de vie : sans import, Terraform tenterait de créer un nouveau réseau, avec un risque de doublon ou d'erreur. Importer le réseau comme resource le placerait dans le périmètre du projet, y compris pour une éventuelle destruction.

Q2 - Quelle frontière entre Terraform et Ansible ?

Terraform provisionne les objets OpenStack : VM, Security Group, règles, FIP et association Neutron. Ansible configure le système invité : paquets, DNS, fichiers et service K3s. Ainsi, associer une FIP relève de l'API OpenStack ; écrire la configuration K3s relève de la VM. Helm déploie ensuite l'application. Cette séparation est un choix d'architecture : les outils possèdent des mécanismes de recouvrement.

Q3 - Pourquoi un StatefulSet pour PostgreSQL ?

Première différence : le pod conserve une identité stable, postgres-0, avec le Service headless postgres. Deuxième différence : volumeClaimTemplates associe le PVC data-postgres-0 à cette identité et le réutilise lors du remplacement du pod. Un Deployment gère des pods interchangeables sans ce mécanisme de PVC par réplique. Le stockage local-path survit au remplacement du pod, mais pas à la destruction du disque de la VM.

Q4 - Pourquoi un tunnel SSH plutôt qu'ouvrir 6443 ?

Fermer 6443 dans le SG évite d'exposer l'API d'administration Kubernetes au réseau extérieur.
SSH transporte les requêtes du port local 6443 vers 127.0.0.1:6443 dans la VM, via le port 22.
L'accès reste chiffré et authentifié par SSH, avec les contrôles TLS et les credentials Kubernetes.

Q5 - Que subsiste-t-il après terraform destroy ?

Le socle du TD1 subsiste : projet/utilisateur, réseau asl-net, sous-réseau, routeur, réseau public, image, flavor et paire de clés. Ces objets sont lus par des data sources ou sont hors de ce projet. Terraform détruit ses 7 ressources : VM, SG, 3 règles, FIP et association. Préserver le socle partagé évite de perturber les autres travaux. Le port créé avec la VM suit normalement sa suppression.

Périmètre des preuves et travail restant

La destruction de 7 ressources est un résultat attendu, sans capture fournie. Après les preuves, désinstaller les releases Helm, supprimer les PVC et le namespace, puis exécuter terraform destroy. Le cycle complet du bonus visites (interface, rollout et rollback) n'est pas attesté. Aucune nouvelle exécution DevStack n'est revendiquée dans ce rapport.

Sources : énoncé TD2 v2 (36 pages), captures de l'étudiante (17 pages) et dépôt oumaymaennamous/td2-infra-as-code. Les preuves complètes et les réponses détaillées sont accessibles dans docs/preuves.md et docs/reflexion.md.